P1 SO: libreta

Oct 7, 2026 · @Julian

Apuntes cortos para la defensa: unas tres líneas por cada cosa, para copiar en la libreta.

Idea general

Qué es. Un shell es un programa que repite: escribe el prompt `->`, lee la línea con `fgets`, la parte en palabras con `TrocearCadena` y ejecuta el comando. Todo está en `p1.c` y se compila con `gcc -Wall p1.c -o p1`.

`trozos`. `TrocearCadena` usa `strtok`, que no copia nada: pone `\0` donde había espacios y cada `trozos[i]` apunta a una palabra dentro de `cadena`. Al final queda un `NULL`, así que `trozos[1] == NULL` significa "sin opciones". Mide `MAX_TEXT/2 + 1` porque el peor caso (`a a a...`) son 1024 palabras más el `NULL`.

El main. Llama a `InicializarAbiertos` y entra en un `while (1)`: prompt y `fflush(stdout)` para que salga ya; `fgets` (si da `NULL` es Ctrl+D y se sale con `break`); trocear (si da 0 la línea está vacía y se hace `continue`); y buscar el comando en la tabla.

La tabla de comandos. `comandos[]` es un array de `struct COMANDO` con nombre, puntero a la función y texto de ayuda, terminado en `{NULL, NULL, NULL}`. El main la recorre con un `for` y, si el nombre coincide con `trozos[0]`, llama a `comandos[i].funcion(trozos)`; si llega a `NULL`, "Comando no reconocido". Por eso todas las `Cmd_` reciben `char *trozos[]`, aunque no lo usen (`(void) trozos;`), y arriba están sus prototipos.

Puntero a función. Es una variable que guarda dónde está el código de una función: `Cmd_date` sin paréntesis es su dirección, y con paréntesis se ejecuta. Comando nuevo = función + prototipo + una fila en la tabla, sin tocar el main.

Grupo 1: información

`authors [-l|-n]`. Sin opción saca nombres y logins; con `strcmp` mira si es `-n` (solo nombres) o `-l` (solo logins). `strcmp` devuelve 0 cuando los dos textos son iguales.

`pid [-p]`. `getpid()` da el pid del shell y `getppid()` el de su padre (la terminal). La condición es `trozos[1] != NULL && strcmp(...)`: primero se mira que exista, porque `strcmp` con `NULL` rompe el programa, y `&&` no evalúa la segunda parte si la primera es falsa.

`date [-d|-t]`. `time(NULL)` da los segundos desde el 1/1/1970; `localtime` los pasa a una `struct tm` con día, mes, año... en hora local; y `strftime` lo escribe como texto (`%d/%m/%Y` y `%H:%M:%S`, que vienen en `man strftime`). Sin opción sale la hora y luego la fecha.

`chdir [dir]`. Sin argumento, `getcwd` da la carpeta actual. Con argumento, `chdir` cambia; si devuelve -1, `perror` muestra el motivo, que el sistema dejó en `errno`. Tiene que ser interno porque cada proceso tiene su propia carpeta actual.

`sysinfo`. `uname(&info)` rellena una `struct utsname` con equipo, procesador, sistema y kernel. Se le pasa `&info` para que pueda escribir en nuestra variable. No usa `trozos`, por eso lleva `(void) trozos;`.

`help [cmd]`. Recorre la misma tabla de comandos: sin argumento imprime todos los nombres, y con argumento busca esa fila e imprime `nombre: ayuda`. Si el bucle acaba sin encontrarlo, "no encontrado".

`exit` y `bye`. Las dos filas apuntan a `Cmd_fin`, que hace `exit(0)`: termina el programa al momento, y el 0 significa que acabó bien.

Grupo 2: ficheros abiertos

Descriptor. Al abrir un fichero, el sistema da un número pequeño (el descriptor), que es su posición en la tabla de abiertos del proceso. Todo proceso hereda de su padre el 0 (teclado), el 1 (pantalla) y el 2 (errores), y `open` da el libre más bajo, así que el primero es el 3.

La lista. Es un array global de `struct FICHERO` (nombre y modo) donde la posición es el descriptor: `abiertos[3]` es el df 3, y un nombre vacío significa cerrado. Empieza todo a 0 por ser global, y `InicializarAbiertos` mete 0, 1 y 2 con su modo, que se pregunta con `fcntl(df, F_GETFL)`. Tiene que coincidir con `lsof -p <pid>`.

Los modos. `O_CREAT`, `O_APPEND`... son bits: con `|` se juntan y con `&` se comprueba si uno está. `O_RDONLY`, `O_WRONLY` y `O_RDWR` ocupan los dos últimos bits, por eso se separan con `& O_ACCMODE`. `ModoTexto` los pasa a palabras.

`LeerDescriptor`. Pasa el texto del df a número: comprueba que empiece por una cifra, usa `atoi` y mira que no se salga del array. Si no vale, devuelve -1. Evita un segmentation fault con `close 99999` y que `close patata` cierre el 0.

`listopen`. Recorre el array y muestra cada descriptor abierto con su nombre, su modo y su posición, que se saca con `lseek(df, 0, SEEK_CUR)` (moverse 0 devuelve dónde está).

`open fich modos`. Traduce cada modo (`cr`, `rw`, `ap`...) y los junta con `modo |=`. Llama a `open(fich, modo, 0777)` (0777 son los permisos si se crea), y si va bien guarda nombre y modo en `abiertos[df]` con `strncpy`. Sin argumentos, lista los abiertos.

`close df`. Comprueba el df, lo cierra con `close` y deja su nombre vacío. Si ya estaba cerrado: `Imposible cerrar descriptor: Bad file descriptor`.

`dup df`. `dup` da otro descriptor para el mismo fichero abierto, así que los dos comparten la posición (como dos llaves de la misma puerta). Se guarda como `dup 3 (nombre)` con `sprintf`, como en el código de los profesores, y su modo se pregunta con `fcntl`.

`lseek df pos ref`. Mueve la posición contando desde el principio (`SEEK_SET`), desde donde está (`SEEK_CUR`) o desde el final (`SEEK_END`). La posición se pasa con `atol` y puede ser negativa.

`readstr df cont`. Lee como mucho `cont` bytes con `read` en un buffer fijo de `MAX_TEXT`, pone el `\0` detrás de lo leído y lo imprime. `read` devuelve cuánto leyó de verdad.

`writestr df texto`. Escribe con `write` cada palabra desde `trozos[2]`, con un espacio entre medias, porque `TrocearCadena` las había separado. Si va bien no dice nada.

Grupo 3: crear y borrar

Sin nombres. Los cuatro comandos enseñan la carpeta actual con `MostrarDirectorioActual` (que usa `getcwd`), como el shell de referencia.

`makefile nombre`. Crea un fichero vacío con `open(nombre, O_CREAT | O_EXCL | O_WRONLY, 0777)`. `O_EXCL` hace que falle si ya existe, para no machacarlo. Se cierra enseguida y no se mete en la lista de abiertos.

`makedir nombre`. Crea una carpeta con `mkdir(nombre, 0755)`: el dueño puede todo y los demás, leer y entrar.

`delete n1 n2...` (`BorrarUno`). Con `lstat` mira qué es cada nombre: si `S_ISDIR` dice que es carpeta, `rmdir` (solo si está vacía); si no, `unlink`. Si uno falla, avisa con `strerror(errno)` y sigue con el siguiente.

`deltree n1 n2...` (`BorrarArbol`, recursivo). Si es una carpeta, la recorre con `opendir`/`readdir`, se salta `.` y `..`, monta cada ruta `carpeta/cosa` con `snprintf` y **se llama a sí misma** con ella. Al final hace `closedir` y borra la carpeta, que ya está vacía.

El `.` y el `..`. Toda carpeta los tiene: `.` es ella misma y `..` la de arriba. Si `deltree` no se los saltara, subiría y lo borraría todo.

`lstat` y no `stat`. Con un enlace simbólico, `stat` mira a dónde apunta y `lstat` el enlace en sí. Así `deltree` de un enlace a una carpeta borra solo el enlace y no vacía la carpeta de destino.

Grupo 4: listar

Las opciones. `struct OPCIONES` tiene un campo por opción (`largo`, `link`, `acc`, `hid`, `reca`, `recb`), a 1 si se ha puesto. `LeerOpciones` los pone a 0, lee las opciones desde `trozos[1]` y devuelve dónde empiezan los nombres. Recibe un puntero, por eso se usa `op->largo` (es lo mismo que `(*op).largo`).

Los permisos. `LetraTF` y `ConvierteModo` son del código de los profesores: miran los bits de `st_mode` y escriben el tipo y los permisos como `ls -l` (`drwxr-xr-x`), incluidos los especiales `s` y `t`.

`InfoFichero`. Hace `lstat` y, sin `-long`, escribe tamaño y nombre. Con `-long`: fecha (`st_mtime`, o `st_atime` con `-acc`), enlaces, inodo, dueño, grupo, permisos, tamaño y nombre en una línea. Con `-link` y si es un enlace, añade `-> destino` con `readlink`, que no pone el `\0`.

Usuario sin dueño. `getpwuid` y `getgrgid` pasan el número de usuario y de grupo a su nombre. Si el usuario no existe devuelven `NULL`, y usar `pw->pw_name` daría segmentation fault, así que escribimos `??????`.

`listfile`. Lee las opciones y llama a `InfoFichero` con cada nombre. Si es una carpeta, habla de la carpeta en sí, no de lo de dentro.

`list`. Si el nombre no es una carpeta, igual que `listfile`. Si lo es, `ListarDirectorio` escribe `**********carpeta` y llama a `InfoFichero` con cada cosa de dentro; las que empiezan por `.` solo salen con `-hid`.

Recursividad (`-reca` y `-recb`). `RecorrerSubdirectorios` busca las subcarpetas (sin `.` ni `..`) y llama a `ListarDirectorio` con cada una, y esta vuelve a llamar a la primera. `-recb` entra en las subcarpetas antes de listar la carpeta, y `-reca`, después. Con `lstat`, un enlace a una carpeta no se recorre, así que no hay bucles infinitos.

Cambios que hicimos

Simplificación. Para poder explicarlo mejor: `atoi` en vez de `strtol` (comprobando a mano que empiece por una cifra), buffer fijo en `readstr` en vez de `malloc` (no hay memoria dinámica, así que no puede haber fugas), `sprintf` en `dup` como en el código de los profesores, y tipos normales (`int`, `long`).

Código de ayuda de los profesores. Usamos tal cual `LetraTF` y `ConvierteModo`, el nombre `dup 3 (fichero)` y el modo del `dup` preguntado con `fcntl`.

Shell de referencia. Lo ejecutamos y copiamos sus formatos y mensajes: `Pid de shell`, hora y luego fecha, `sysinfo` como `equipo (procesador), OS: ...`, `help` como `nombre: ayuda`, los mensajes `Imposible...` y `Parametros incorrectos`, el offset en `listopen` y los formatos de `listfile` y `list`. Para meter el motivo de un error en un `printf` usamos `strerror(errno)`.

Diferencias a propósito. `authors` lleva nuestros nombres reales; `listopen` solo enseña los abiertos; y `close patata` no cierra el 0, cosa que sí pasa en la referencia porque usa `atoi` sin comprobar.

Probar y entregar

Probar. Compilar con `gcc -Wall -Wextra p1.c -o p1`, sin ningún aviso. Ejecutar `./p1` y probar también lo raro (líneas vacías, `close patata`, `readstr 3 -2`, `chdir noexiste`), porque un segmentation fault es un 0. `valgrind ./p1` tiene que decir "no leaks are possible" y "0 errors", y `lsof -p <pid>` tiene que coincidir con `listopen`.

Errores al compilar. Muchos errores en la misma línea suelen ser un paréntesis o un `;`. El número detrás de `p1.c:` es la línea (Ctrl+I en el editor para ir a ella).

Entregar. `p1.zip` (p minúscula) con dentro solo la carpeta `P1` (P mayúscula) y `p1.c`. Se hace en la terminal: `mkdir -p entrega/P1`, `cp p1.c entrega/P1/`, `cd entrega`, `zip -r p1.zip P1`, y se comprueba con `unzip -l p1.zip`. Sin ejecutables ni ficheros de prueba, y lo sube solo uno de los dos, con margen antes del viernes 9 a las 23:00.